
Kubernetes Pods – Interview Questions & Answers

♦ Q1: What is a Pod in Kubernetes?

Answer:

A Pod is the smallest deployable unit in Kubernetes, which can contain one or more containers. All containers in a Pod share the same network namespace and storage volumes.

♦ Q2: Can multiple containers run in the same Pod? Why?

Answer:

Yes. Containers in a Pod can run together if they are tightly coupled (e.g., a main app and a sidecar). They share the same IP and volumes, making inter-container communication fast and simple.

♦ Q3: What are typical use cases for multi-container Pods in production?

Answer:

- Logging agents
 - Monitoring sidecars
 - Proxies (e.g., Envoy in Istio)
 - Helper containers for config reloading or secret watching
-

♦ Q4: How do you debug a Pod that's stuck in **CrashLoopBackOff**?

Answer:

- Check logs: `kubectl logs <pod-name>`
 - Describe Pod: `kubectl describe pod <pod-name>`
 - Inspect events for OOM, readiness probe failures, image pull errors, etc.
 - Use `kubectl exec` to inspect the filesystem (if possible)
-

♦ Q5: How do you ensure high availability for Pods?

Answer:

Do **not** create Pods directly. Use **Deployments** or **StatefulSets** with **replicas** > 1 to ensure automatic healing and scheduling across nodes.

 ReplicaSets – Interview Questions & Answers

♦ Q6: What is a ReplicaSet?

Answer:

A ReplicaSet ensures that a specified number of pod replicas are running at any given time. It replaces failed Pods but does not support rolling updates directly.

♦ Q7: When would you use a ReplicaSet directly in production?

Answer:

Almost never. ReplicaSets are managed by **Deployments**. Managing them directly removes support for rolling updates and version control.

♦ Q8: How does a ReplicaSet select Pods?

Answer:

Via **label selectors**. If a Pod matches the ReplicaSet's label selector, it is managed by that ReplicaSet—even if the Pod was created manually.

♦ **Q9: Can multiple ReplicaSets manage the same Pod?**

Answer:

No. While multiple ReplicaSets can **match** a Pod via label selectors, only **one** will claim ownership. This can cause unpredictable behavior and should be avoided.

 Deployments – Interview Questions & Answers

♦ **Q10: What is a Deployment in Kubernetes?**

Answer:

A Deployment manages ReplicaSets and provides **declarative updates** to Pods. It supports versioning, rollback, and rollout strategies.

♦ **Q11: How does a Deployment perform a rolling update?**

Answer:

Kubernetes creates a new ReplicaSet with the updated spec, gradually scales it up while scaling the old one down, based on **maxSurge** and **maxUnavailable** settings.

♦ **Q12: How do you roll back a failed Deployment?**

Answer:

`kubectl rollout undo deployment <deployment-name>`

You can also check history using:

```
kubectl rollout history deployment <deployment-name>
```

♦ **Q13: What's the difference between a Deployment and a StatefulSet?**

Answer:

Feature	Deployment	StatefulSet
Pod Identity	Ephemeral	Stable network identity
Volume	Shared or none	Persistent, per Pod
Use Case	Stateless apps	Databases, Kafka, etc.

♦ **Q14: What are some common production issues with Deployments?**

Answer:

- Misconfigured rolling update settings (`maxUnavailable` too high)
 - Probes (liveness/readiness) causing early restarts
 - Resource limits not set, leading to node pressure
 - Not specifying `strategy` or version pinning in image tags
-

♦ **Q15: How do you pause a Deployment rollout?**

Answer:

```
kubectl rollout pause deployment <deployment-name>
```

You can later resume with:

```
kubectl rollout resume deployment <deployment-name>
```

♦ **Q16: How do you implement canary deployments with Deployments?**

Answer:

Manually or with tools like Argo Rollouts or Flagger. You create two Deployments:

- One with 90% traffic (old version)
 - One with 10% (new version)
Update service selectors accordingly.
-

♦ **Q17: How do resource requests and limits affect Pod scheduling?**

Answer:

- **Requests:** Scheduler uses them to place Pods on nodes.
 - **Limits:** Enforced at runtime. If a container exceeds memory limits, it may be OOM-killed.
-

♦ **Q18: How do you automate scaling a Deployment?**

Answer:

Use a **HorizontalPodAutoscaler (HPA)**:

```
kubectl autoscale deployment <name> --cpu-percent=70 --min=2 --max=10
```

♦ **Q19: How do readiness and liveness probes differ in production?**

Answer:

- **Readiness:** Controls traffic routing. If it fails, the Pod is removed from the Service.
 - **Liveness:** Restarts the container if it becomes unhealthy.
-

♦ Q20: How do you prevent downtime during Deployment updates?

Answer:

- Set proper `readinessProbe`
 - Use `rollingUpdate` strategy with conservative `maxUnavailable`
 - Avoid breaking changes (e.g., DB schema changes) mid-rollout
 - Monitor rollouts using `kubectl rollout status`
-